

Lenguaje de Programación

JavaScript

Avanzado

(Clases)

Introducción a Web Components

Contenido

1.	Clases en JavaScript	3
1.1.	Prototipos en ES5 vs Clases en ES6	3
1.2.	Variables y métodos privados	4
1.3.	Métodos estáticos	6
1.4.	Herencia de clases	6
2.	Módulos	8
2.1.	Exportación de módulos	8
2.2.	Importación de módulos	9
2.3.	Reglas para el uso de módulos	10
3.	Web Components nativos	11
3.1.	Custom Elements	12
3.3.	Shadow DOM	14
3.1	HTML Templates	16
3.2	ES Modules	17

1. Clases en JavaScript

JavaScript sigue siendo un lenguaje de programación basado en prototipos. Esto quiere decir que todo lo que no sean tipos primitivos (undefined, null, boolean, number o string) son objetos y pueden ser usados como prototipos para crear otros objetos.

Consideremos la siguiente función Persona:

```
function Persona(nombre, edad) {  
  this.nombre = nombre;  
  this.edad = edad;  
}
```

El prototipo de esta función es `Persona.prototype` y podemos añadirle más métodos o propiedades de las que tiene originalmente:

```
Persona.prototype.toString = function() {  
  return "[Persona: " + this.nombre + "]";  
}  
  
Persona.prototype.saludar = function() {  
  console.log("Hola mi nombre es: " + this.nombre);  
}  
  
var p = new Persona("John", 33);  
console.log(p.nombre); // propiedad del objeto  
console.log(p.saludar());
```

No obstante, las clases en JavaScript son en realidad solo una forma más bonita de escribir los prototipos de manera que sea más sencillo para la gente que llega de otros lenguajes el acostumbrarse a JavaScript. Hay muchas de las características de ES6 que no son más que una forma más sencilla/bonita de hacer algo que siempre hemos hecho.

1.1. Prototipos en ES5 vs Clases en ES6

Veamos primero un ejemplo que debería resultarnos familiar:

```
function Empleado (nombre, apellidos) {  
  this.nombre = nombre;  
  this.apellidos = apellidos;  
  this.sueldo = 16000;  
  this.trabajoRealizado = 0;  
}  
  
Empleado.prototype.nombreCompleto = function() {  
  return this.nombre + ' ' + this.apellidos;  
};  
  
Empleado.prototype.trabajar = function() {  
  this.trabajoRealizado += 100;  
};
```

```
Empleado.prototype.subirSalario = function(cantidad) {
  cantidad = cantidad || 1000;

  this.salario += cantidad;
};

var empleado = new Empleado('Antonio', 'Laguna');
console.log(empleado.nombreCompleto()); // Antonio Laguna
```

Precioso. ¿Cómo quedaría con una clase de ES6? La sintaxis no varía mucho excepto que usamos la palabra clave **class** en vez de **function**. Los métodos de la clase se declaran usando una sintaxis mucho más corta sin usar la palabra **prototype**.

```
class Empleado {
  constructor(nombre, apellidos) {
    this.nombre = nombre;
    this.apellidos = apellidos;
    this.salario = 16000;
    this.trabajoRealizado = 0;
  }

  nombreCompleto() {
    return this.nombre + ' ' + this.apellidos;
  }

  trabajar() {
    this.trabajoRealizado += 100;
  }

  subirSalario(cantidad) {
    cantidad = cantidad || 1000;
    this.salario += cantidad;
  }
}

let empleado = new Empleado('Antonio', 'Laguna');
console.log(empleado.nombreCompleto()); // Antonio Laguna
```

Si te fijas, es muy parecida a la notación que usamos al crear un objeto literal. No obstante, una diferencia bastante notable es que los puntos y coma entre métodos **están prohibidas**, lo cual crea un poco de confusión. No obstante, en líneas generales, el código se ve bastante mejorado entre ES5 y ES6.

CUIDADO! Cuando estamos creando métodos en una clase, puede ser habitual que queramos darle a un método el mismo nombre que a una propiedad, por ejemplo, en los métodos **set** y **get**. En estos casos se nos devolverá un error y podremos llenar la pila de memoria de los navegadores porque se realizan llamadas recursivas. Para evitar esto, deberemos evitar poner los mismos nombres.

Una solución sería la de tomar la costumbre de empezar el nombre de todas las propiedades por el carácter “_”. Otra solución sería la de empezar el nombre de los métodos por mayúscula.

1.2. Variables y métodos privados

Desde la implementación 12 de JavaScript (2021), podremos añadir a nuestras clases Javascript **tanto propiedades como métodos privados** con lo que aumentamos el encapsulamiento de las clases y privacidad de los datos. Esto quiere decir que solamente podremos acceder a estas propiedades o métodos desde otro método de la propia clase.

La forma de hacer privados tanto atributos como métodos será añadir # delante de su nombre. Veamos algunos ejemplos:

Atributo y getter/setter públicos:

```
class Test {
  const _value = 0

  get value() { return this._value }

  set value(newValue) { this._value = newValue }
}
```

Atributo y getter/setter privados:

```
class Test {
  const #_value = 0

  get #value() { return this.#_value }

  set #value(newValue) { this.#_value = newValue }
}
```

Otro ejemplo:

```
class Usuario {
  #nombre;
  constructor(nombre) {
    this.#nombre = nombre;
  }

  #apellidos(apellido) {
    return this.#nombre + " " + apellido;
  }

  mostrar(apellido) {
    console.log(this.#apellidos(apellido));
  }
}

const usuario = new Usuario("Pepe");
console.log(usuario.#nombre); // Error!
usuario.mostrar("Gómez"); // Pepe Gómez
```

Hay que tener en cuenta que las propiedades que sean privadas deben declararse fuera del constructor antes de poder usarlas.

1.3. Métodos estáticos

Es habitual que las clases tengan métodos *estáticos*. Por ejemplo, la *clase* `String`, tiene métodos de instancia como `replace`, `trim` o `toUpperCase` y métodos estáticos como `String.fromCharCode()` o `String.fromCodePoint()`.

Vamos a ver cómo añadir un método estático a nuestra clase `Empleado`.

```
class Empleado {
  constructor(nombre, apellidos) {
    this.nombre = nombre;
    this.apellidos = apellidos;
    this.salario = 16000;
  }

  static tieneMasSuelto(empleado1, empleado2) {
    return empleado1.salario > empleado2.salario;
  }

  nombreCompleto() {
    return this.nombre + ' ' + this.apellidos;
  }

  trabajar() {
    this.trabajoRealizado += 100;
  }

  subirSalario(cantidad) {
    cantidad = cantidad || 1000;

    this.salario += cantidad;
  }
}

let juan = new Empleado('Juan', 'Gómez');
let pepe = new Empleado('Pepe', 'González');
pepe.salario = 20000;

alert(Empleado.tieneMasSuelto(pepe, juan)); // true
```

1.4. Herencia de clases

Siempre que salen las clases a relucir, existe el tema de la herencia, algo que es bastante útil y que, hasta la versión ES5, era bastante complejo de realizar en JavaScript.

Vamos a crear la clase `Jefe` que *extiende* a la clase `Empleado`. Un `Jefe` puede despedir a un empleado así que vamos a crear este método:

```
class Jefe extends Empleado {
  despedir(empleado) {
    alert(empleado.nombreCompleto() + ' estás despedido!');
    empleado.despedido = true;
  }
}
```

```
}

let javi = new Jefe('Javi', 'Perales');
let pedro = new Empleado('Pedro', 'Ortiz');

javi.despedir(pedro); // 'Pedro Ortiz estás despedido!';
pedro.despedir(javi); // "TypeError: pedro.despedir is not a function"
```

La clase **Jefe** contiene todos los métodos y el constructor que contiene la clase **Empleado**. No obstante, es posible sobrescribir un método cualquiera. Por ejemplo:

```
class Jefe extends Empleado {
  despedir(empleado) {
    alert(empleado.nombreCompleto() + ' estás despedido!');
    empleado.despedido = true;
  }

  trabajar() {
    super.trabajar();
    this.trabajoRealizado += 50;
  }
}
```

Como ves, hemos implementado el método **trabajar** también en la clase **Jefe**. Gracias a la palabra clave **super**, podemos acceder a la clase **Empleado** que estamos extendiendo. Es muy habitual que, al reimplementar un método, llamemos al método base para realizar la misma función y añadir algo adicional, *añadido*.

Lo más habitual, no obstante, es que sobrescribamos el constructor de una clase, y únicamente tendremos que usar **super()** para llamar al constructor de la clase extendida. Es importante destacar que **es necesario** pasar los argumentos que la clase necesite para hacerla funcionar:

```
class Jefe extends Empleado {
  constructor(nombre, apellidos) {
    super(nombre, apellidos);
    this.salario = 30000;
  }

  despedir(empleado) {
    alert(empleado.nombreCompleto() + ' estás despedido!');
    empleado.despedido = true;
  }

  trabajar() {
    super.trabajar();
    this.trabajoRealizado += 50;
  }
}
```

Como ves aquí, la clase **Jefe** ahora tiene un salario base más alto que la clase **Empleado**.

CUIDADO! Si en un método o en el constructor vamos a acceder a la clase padre mediante **super()**, tendremos que hacer esta llamada antes de acceder a las propiedades del objeto.

2. Módulos

En ES6 se introduce el concepto de módulo que no es otro que permitir trocear el código en distintos ficheros, de forma que se permita la exportación de clases, funciones, variables o constantes e importarlos desde otro módulo.

Para poder usar módulos en tu aplicación web, debes añadir el texto `type="module"` en la etiqueta `<script></script>` de tu fichero .js principal. Esta propiedad ya incluye la ejecución en diferido “defer”, luego podrás ya quitarla.

Véase el siguiente ejemplo:

```
//---Fichero lib.js -----  
  
export const sqrt = Math.sqrt;  
  
export function square(x) {  
    return x * x;  
}  
  
export function diag(x, y) {  
    return sqrt(square(x) + square(y));  
}
```

Ahora podremos llamar a estas funciones, variables, constantes, clases, ... desde otro fichero .js. Para ello usaremos la sintaxis siguiente:

```
//----- main.js -----  
  
import { square, diag } from 'lib.js';  
  
console.log(square(11)); // 121  
console.log(diag(4, 3)); // 5
```

En caso de que nuestro fichero tuviera muchas funciones y quisiéramos importarlas todas, pero no tener que declararlas una a una, podemos usar un *alias*:

```
import * as Lib from './circle.js';  
  
console.log( 'El cuadrado de 4 es ' + Lib.square(4));
```

Antes de proseguir debes tener clara la diferencia entre **módulos** y **web workers**. El primero es simplemente una forma de dividir la funcionalidad JavaScript entre varios ficheros. El segundo, permite que varios ficheros trabajen de forma paralela y se puedan comunicar entre ellos.

2.1. Exportación de módulos

Por defecto, un fichero Javascript no tiene módulo de exportación si no se usa un *export* al menos una vez. Existen varias formas de exportar código mediante la palabra clave *export*:

Forma	Descripción
<code>export { name };</code>	Añade el elemento <i>name</i> al módulo de exportación.
<code>export { n1, n2, n3... };</code>	Añade los elementos indicados (n1, n2, n3...) al módulo de exportación.
<code>export * from './file.js';</code>	Añade todos los elementos del módulo file.js al módulo de exportación.
<code>export declaration;</code>	Declara una variable, función o clase y la añade al módulo de exportación.
<code>export default declaration;</code>	Declara una función o clase y la añade al módulo de exportación.

Es posible renombrar los elementos sobre la marcha utilizando *as* seguido del nuevo nombre.

`export * as utils from './utils.js'`

Además, si se indica ***default*** como nuevo nombre, ese elemento será la exportación por defecto. Sólo puede haber una exportación por defecto por fichero.

Hay que tener en cuenta que, en el caso de utilizar una exportación ***por defecto*** en una declaración, no es posible utilizar *var*, *let* o *const*. Tampoco es posible usar *export* dentro de funciones, bucles o contextos específicos.

2.2. Importación de módulos

Para importar módulos en otro fichero .js, usaremos ***import***. Existen varias formas de importar en un módulo:

Forma	Descripción
<code>import nombre from './file.js';</code>	Importa sólo el elemento por defecto (<i>default</i>) de file.js en nombre.
<code>import { nombre } from './file.js';</code>	Importa sólo el elemento nombre de file.js.
<code>import { n1, n2.. } from './file.js';</code>	Importa los elementos indicados desde file.js.
<code>import * as obj from './file.js';</code>	Importa todos los elementos de file.js en el objeto obj.

Forma	Descripción
<code>import './file.js';</code>	No importa elementos, pero ejecuta el código de <code>file.js</code> .

Recuerda que, al igual que con la exportación, también puedes renombrar elementos utilizando **as** seguido del nuevo nombre:

```
import * as utils from './utils.js'
```

En el primer caso, importamos el elemento por defecto desde el módulo `file.js` y lo guardamos en la variable *nombre*. En el segundo y tercer caso, importamos los elementos indicados en el interior de las llaves, desde el módulo `file.js`.

2.2.1. Imports dinámicos **Nuevo!**

Las importaciones dinámicas en JavaScript dan la opción de importar archivos JS de forma dinámica como módulos en su aplicación de forma nativa. Esta función ayuda a enviar código de solicitud bajo demanda. Esta característica permite importar un módulo y no ensuciar el espacio de nombres global. También se puede cargar código condicionalmente en un bloque if-else si se desea. Véase el ejemplo siguiente:

```
if (myCondition) {  
  const module = await import('./dynamicmodule.js')  
  module.addNumbers(3, 4, 5) // Método de dynamicmodule.js  
}
```

2.2.2. Import.meta **Nuevo!**

Nos dará metainformación de un módulo. Considera un módulo, `module.js`:

```
<script type="module" src="module.js"></script>
```

Puedes acceder a la metainformación con el objeto `import.meta`:

```
console.log(import.meta) //{url: "file://home/user/module.js"}
```

Devuelve un objeto con una propiedad de URL que indica la URL base del módulo. Esta será la URL de la que se obtuvo la secuencia de comandos (para secuencias de comandos externas) o la URL base del documento que lo contiene (para secuencias de comandos en línea).

2.3. Reglas para el uso de módulos

- Si queremos utilizar *import* y *export* desde el navegador directamente, deberemos añadir los archivos con módulos con la etiqueta `<script>` utilizando el atributo **type="module"**. Estas etiquetas de módulos se cargan en diferido, o lo que es lo mismo, como si fueran un `<script defer>`.
- Por norma general, a los archivos Javascript con módulos se les suele indicar la extensión **.mjs**. Aunque también se podría hacer especificando otra extensión como **.es6** o **.js**.
- Se aconseja utilizar las rutas UNIX en los *export* e *import*, ya que son las que tienen mejor soporte, tanto en navegadores como en *NodeJS*. También se pueden indicar rutas absolutas.

3. Web Components nativos

Son componentes nuevos que creamos para HTML totalmente personalizados y usando etiquetas propias. Tienen sus propias propiedades, eventos y pueden ofrecer una nueva paleta de componentes reutilizables en forma de librerías .html o .js.

Suponen una revolución en el mundo del desarrollo Web en términos de **reutilización y encapsulación de código cliente**, ya que permiten encapsular código HTML, CSS y JavaScript, de forma que no puede ser afectado por el código de la página que lo incluye. Son fácilmente reconocibles porque **siempre deben llevar un guion en el nombre**.

Podemos usarlos para **encapsular el uso de APIs de JavaScript** externas y para **extender el comportamiento de elementos nativos**. Otro uso es el de **añadir marcado semántico**, por ejemplo, supongamos que para los productos de una tienda online tenemos el siguiente Web Component:

```
<producto-tienda precio="99" precioAntes="88" href="/enlace-producto.html">
  <div class="nombre">Nombre producto</div>
  <div class="descripcion">Descripción producto</div>
</producto-tienda>
```

En el ejemplo anterior hemos definido un web component con las propiedades *precio*, *precioAntes* y *href*, y está compuesto por dos *divs*.

¿Qué vamos a necesitar para implementar nuestros Web Components?

- Custom Elements.
- HTML Templates.
- Shadow DOM.
- ES Modules.

La aproximación básica para implementar un componente web, generalmente es la siguiente:

1. Crear una clase en JavaScript en la cual especificaremos la funcionalidad del componente web.
2. Registrar el nuevo elemento personalizado utilizando el método:

`window.customElementRegistry.define(<nombre-webComponent>, <Clase>)`

, pasándole el nombre del elemento a ser definido, la clase en la cual su funcionalidad está especificada y, opcionalmente, un objeto con el nombre de otro elemento para extender su funcionalidad. Ahora, solo acepta un string con la palabra *extends*. Por ejemplo:

`customElements.define('word-count', WordCount, { extends: 'p' });`

3. **Opcional.** Si se requiere, adjuntar un shadow DOM al elemento personalizado usando el método **`Element.attachShadow()`**. Esto permitirá añadir elementos hijos, oyentes de eventos, etc., al shadow DOM utilizando métodos normales del DOM.
4. **Opcional.** Si se requiere, definir una plantilla HTML utilizando **`<template>`** y **`<slot>`**. Nuevamente se usarán métodos regulares del DOM para clonar la plantilla y acoplarla al shadow DOM.
5. Utilizar los componentes personalizados en la página web cuando se desee, como se utilizaría cualquier otro elemento HTML.

3.1. Custom Elements

Es una API de JavaScript que permite definir elementos personalizados y su comportamiento, para poder ser utilizado como se desee en la interfaz del usuario. Consistirá en la implementación de una clase en JavaScript con una serie de métodos predefinidos por la API.

Ofrece la posibilidad de extender el lenguaje HTML con elementos propios. Por ejemplo:

```
<acme-calendar mode="month" date="2020-11-23" on-select="dateSelected()">
</acme-calendar>
```

`<acme-calendar>` es una nueva etiqueta que definimos nosotros en el documento HTML. En el ejemplo anterior damos valor a sus propiedades *mode* y *date* y, además, ejecutamos la rutina **`dateSelected()`** de JavaScript cuando el calendario se seleccione.

Está estandarizado por W3C en todos los navegadores.

Para crear un Web Component propio haremos lo siguiente en JavaScript:

1. Crearemos una clase con el nombre del componente web, que herede de ***HTMLElement***. En esta clase se definirá el comportamiento de nuestro Web component y tendrá, al menos, los siguientes elementos:
 - **Constructor.**
 - **Método `connectedCallback()`** {} → Se llama cada vez que el elemento se inserta en el DOM. Aquí podemos hacer llamadas AJAX para pedir datos, configurar cosas, etc.
 - **Método `disconnectedCallback()`** {} → para definir que se hace cuando el componente se elimina del árbol DOM.

- **Método `attributeChangedCallback()`** {} → Este método se llamará cada vez que se añadiera, eliminara o que cambiara un atributo del custom element.
- **Método `render()`** {} → para definir qué se hace cuando el componente se tiene que redibujar en la web.

```
class WebComponent extends HTMLElement {
  constructor() {
    // Se debe llamar a super siempre que se sobreescriba el constructor.
    super();
  }

  connectedCallback () {
    this.innerHTML = `
      <style>
        :host {
          --space: 1.5em;
        }
        .wc-div {
          border: 2px dashed orange;
          padding: 5px;
          text-align:center;
        }
        .wc-botton {
          background-color: orange;
          border: 0;
          border-radius: 5px;
          color: white;
          padding: var(--space);
          text-transform: uppercase;
        }
      </style>
      <div class="wc-div">
        <button class="wc-botton">${txtButton}</button>
      </div>`;
  }

  disconnectedCallback () {}
  attributeChangedCallback () {}
  _render () {}
}
```

2. Definiremos el componente para que el navegador pueda reconocerlo.

```
// Agregamos el componenete a los elementos pesonalizados o propios
window.customElements.define("web-component", WebComponent);
```

3. A partir de ahora, podremos usar la etiqueta **<web-component>** en el fichero HTML que enlace con nuestro fichero JavaScript.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title>Web Component</title>
  <script src="js/web-component.js"></script>
</head>
<body>
  <web-component></web-component> <!-- - Web Component-- >
</body>
</html>
```

PULSAR

3.3. Shadow DOM

Es una API de JavaScript para fijar un árbol DOM "sombra" encapsulado a un elemento — que es renderizado por separado del documento DOM principal — y controlando funcionalidad asociada. De esta forma, se pueden mantener características de un elemento en privado, así puede tener el estilo y los scripts sin miedo de colisiones o sin que se “pise” su estilo con otras partes del documento.

Es el DOM dentro de cada elemento DOM. Va a proporcionar al *Web component* un nivel distinto dentro del árbol DOM.

Shadow DOM proporciona:

- **Aislamiento** (ámbitos) para código y estilos CSS.
- **Seguridad**: se encapsula el código Javascript.

Por razones de seguridad, no todas las etiquetas HTML se pueden añadir a un elemento Shadow DOM. Las etiquetas que sí podemos añadir son:

- | | |
|--|-------------|
| • Cualquier custom element con un nombre válido. | • <hn> |
| • <article> | • <header> |
| • <aside> | • <main> |
| • <blockquote> | • <nav> |
| • <body> | • <p> |
| • <div> | • <section> |
| • <footer> | • |

Si te fijas, no podemos añadir una etiqueta [<a>](#) a un Web Component, esto es debido a cuestiones de seguridad.

Para crear un nodo *Shadow* usaremos `Element.attachShadow()` en el constructor de la clase:

```
var shadowroot = element.attachShadow(shadowRootInit);
```

El parámetro `shadowRootInit` puede ser:

- **mode:** especifica el modo de encapsulación para el árbol Shadow DOM. Puede tomar dos valores:
 - **open:** El elemento será accesible desde JavaScript, por ejemplo usando `Element.shadowRoot`.
 - **closed:** Deniega el acceso al nodo o nodos desde JavaScript.
- **delegatesFocus:** Un booleano que especifica el comportamiento del elemento cuando éste, o parte de éste, obtiene el foco al hacer click en él.

```
class WebComponent extends HTMLElement {
  constructor() {
    // Se debe llamar a super siempre que se sobrescriba el constructor.
    super();

    // Creamos un nodo Shadow DOM.
    this.wcRoot = this.attachShadow({mode: 'open'});
  }

  connectedCallback () {
    this.wcRoot.innerHTML = `
    <style>
      :host {
        --space: 1.5em;
      }
      .wc-div {
        border: 2px dashed orange;
        padding: 5px;
        text-align:center;
      }
      .wc-botton {
        background-color: orange;
        border: 0;
        border-radius: 5px;
        color: white;
        padding: var(--space);
        text-transform: uppercase;
      }
    `;
  }
}
```

```
        </style>
        <div class="wc-div">
            <button class="wc-botton">${txtButton}</button>
        </div>`;
    }
}

// Agregamos el componenete a los elementos pesonalizados o propios
window.customElements.define("web-component", WebComponent);
```

3.1 HTML Templates

Los elementos **<template>** y **<slot>** permiten escribir plantillas de marcado que no son desplegadas en la página renderizada. Éstas pueden ser reutilizadas en múltiples ocasiones como base de la estructura de un elemento personalizado.

Son plantillas dentro de HTML que pueden tener huecos. Esos huecos los definimos con la etiqueta **slot**. Por ejemplo, podemos tener un fichero HTML con el template:

```
...
<template id="tarticulo">
    <div class="article">
        <h1> <slot name="title"></slot> </h1>
        <hr>
        <slot name="body"></slot>
    </div>
</template>
...
```

Esta plantilla no es visible, pero cuando la instanciamos se dibujará con el contenido que le demos.

Desde JavaScript podemos acceder de la siguiente forma:

```
class TArticulo extends HTMLElement {
    constructor () {
        super();

        this.ta = this.attachShadow({mode: 'open'});
    }

    connectedCallback () {

        let t = document.querySelector('#tarticulo');
        let instance = t.content.cloneNode(true);
```



```
        this.ta.appendChild(instance);
    }
}
window.customElements.define('t-articulo', TArticulo);
```

3.2 ES Modules

Para poder crear instancias de nuestros Web Components en nuestro HTML, podemos enlazar el código de varias formas. Una forma que ha sido muy usada es añadir en la cabecera el fichero .html con todo el código:

```
<link rel="import" href="components/templates.html">
```

En caso de que lo hagamos así, tendremos que encapsular el **<template>** y el **<script>** dentro de un fichero con el tag **<html>**.

```
<html>

<template id="wc">
  // CSS aplicado a los elementos del template:
  <style>

  </style>

  // Estructura del template:
</template>

<script>
  // Código javascript para el Custom element y shadow DOM:
</script>

</html>
```

Esta forma de enlazar componentes web ya está obsoleta desde febrero de 2020. Una mejor forma de hacerlo sería usando **ES Modules**:

```
<script type="module" src="js/components-min.js"></script>
```

En este fichero, dentro del constructor de la clase que crea el componente web, crearemos también el componente usando métodos de JavaScript para crear nodos: `document.createElement()`, `document.createTextNode()`, sus atributos, estilo, ...

```
class <WebComponent> extends HTMLElement {
```

```
constructor () {
  super();

  this.<shadowElement> = this.attachShadow({mode: 'open'});

  //Creamos el componente:
  ...
  //Creamos y añadimos los nodos que deseemos:
  this.<shadowElement>.appendChild(...);

  //Podemos añadirle eventos:
  this.<shadowElement>.addEventListener('mouseover', e => {
    ...
  });
}

// métodos de la clase: get, set, ...
...

// Atributos que serán observados para actualizar el componente cuando cambien
static get observedAttributes () {
  return [<atributo>, <atributo>, ...] // Array de atributos observados
}

// Si hay cambios: se crea, modifica y olimina, que los actualice.
// Parámetros: nombre del atributo, valor anterior, valor nuevo.
attributeChangedCallback (name, oldValue, newValue) {
  ...
}

// Tareas cuando el componente se inserte en el DOM:
connectedCallback () {
  ...
}
}

if (!customElements.get ('<nombre-html>'))
  customElements.define('<nombre-html>', <WebComponent>);
```

Usar componentes web sin frameworks es útil en proyectos pequeños que no requieren de funcionalidades muy avanzadas. Existen frameworks que ayudan a implementar de una forma más sencilla web components, por ejemplo, **React**, **Vue.js** o **Angular**.

Tenemos un catálogo de componentes web ya creados en www.webcomponents.org

Práctica Web Components:

Crear un componente web que tenga como atributos, **coordenada x**, **coordenada y** y **color**. El componente dibujará un cuadrado en la posición indicada, si no se especifica coordenadas, se dibujará en el centro de la pantalla.